

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ
з лабораторної роботи №4
з дисципліни "Основи Програмної Інженерії"

Виконав
ст. гр. ПЗПІ-25-3
Гладких А.Р.

Прийняв:
В.Гребенюк

Харків 2026

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи: Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Результати Code Review одного групу (Алексєєв І.О. робив code review мого коду):

№	Рядок коду	Проблема	Категорія	Рекомендація
1	8	Issue №1: Dead Code / Unused Parameter	Категорія Code Smell: Flawed Logic / Predictable ID Generation	Видаліть параметр age, оскільки він не використовується в логіці методу. Якщо вікова валідація є необхідною бізнес-вимогою, реалізуйте її через перевірку <code>age < 18</code> (або іншого порогу) та викидання відповідної помилки.
2	74	Issue 2: Predictable ID Generation (раніше використовувався <code>Date.now()</code>).	Категорія Code Smell: Flawed Logic / Predictable ID Generation	<code>Date.now()</code> не гарантує унікальності ідентифікаторів, якщо об'єкти створюються в одному циклі або з високою частотою.

				Використовуйте <code>crypto.random UUID()</code> для забезпечення глобальної унікальності ID.
3	60	Issue 3: Violation of Encapsulation (Feature Envy)	Категорія Design Smell: Violation of Encapsulation	Сервіс прямо маніпулює внутрішнім масивом іншого класу. Перенесіть цю логіку в клас <code>EventSession</code> як метод <code>addParticipant(participant)</code> , щоб він сам контролював додавання учасників та дотримання правил місткості.
4	39	Issue 4: Violation of Single Responsibility Principle (SRP)	Категорія Design Smell: Violation of Single Responsibility Principle (SRP)	Клас бере на себе занадто багато обов'язків: реєстрація учасників, створення турів та фільтрація даних. Розділіть їх: наприклад, створіть <code>RegistrationService</code> для роботи з користувачами та <code>EventFactory</code> для створення сесій.

5	45	Issue 5: Lack of Defensive Programming (Type Safety)	Категорія Code Smell: Lack of Defensive Programming (Type Safety)	Методи довіряють вхідним даним без перевірки. Додайте instanceof Participant у методі реєстрації та перевірку, чи є елементи масиву екземплярами EventSession у методі фільтрації, щоб уникнути помилок під час роботи з некоректними типами об'єктів.
---	----	--	---	--

Посилання на PR з коментарями Code Review :
<https://github.com/mik-kotsariev/DeltaPhys/pull/27>

Результати рефакторингу власного коду :
<https://github.com/mik-kotsariev/DeltaPhys/blob/Hladkykh/Lr3/refactored%20code.js>

Операція 1: Відновлення інкапсуляції

БУЛО: `javascript if (!session.hasFreePlaces()) { throw new Error(...); }
const alreadyRegistered =
Array.from(session.registeredParticipants).some(...);
session.registeredParticipants.add(participant); session.capacity--;`

СТАЛО: `javascript session.addParticipant(participant);` *(вся логіка перевірки вільних місць, дублікатів та декременту ємності перенесена*

всередину класу EventSession).

ЧОМУ: Об'єкт має самостійно керувати своїм внутрішнім станом. Пряма маніпуляція сетом `registeredParticipants` та лічильником `capacity` іншого об'єкта з боку сервісу (**Feature Envy / Inappropriate Intimacy**) порушує принципи ООП і робить код крихким при зміні структури даних.

Операція 2: Усунення "Магічних чисел" та умовних обмежень

БУЛО: ````javascript if (title.length < 2) throw new Error("Назва занадто коротка");`

СТАЛО: ````javascript const CONFIG = { MIN_TITLE_LENGTH: 2 }; if (title.length < CONFIG.MIN_TITLE_LENGTH) throw new Error(...);`

ЧОМУ: Винесення жорстко прописаних чисел (літералів) у константи значно підвищує читабельність та спрощує подальший рефакторинг. Якщо бізнес-правила щодо мінімальної довжини назви туру зміняться, достатньо буде оновити константу в одному місці.

Операція 3: Безпечна генерація ідентифікаторів

БУЛО: *(Згідно з первинним зауваженням рев'ю)*

JavaScript

`const id = Date.now();` // або передача невалідного таймстампу

СТАЛО: ````javascript const id = crypto.randomUUID();`

ЧОМУ: Використання `Date.now()` або циклічних лічильників для генерації ID є критичною вразливістю. При високій частоті запитів або асинхронних операціях таймстампи збігаються, що призводить до дублювання ідентифікаторів сесій. Вбудований модуль `crypto.randomUUID()` гарантує криптографічну та глобальну

Звіт регресійного тестування:

```
✓ BVA: Назва занадто коротка (2 ms)

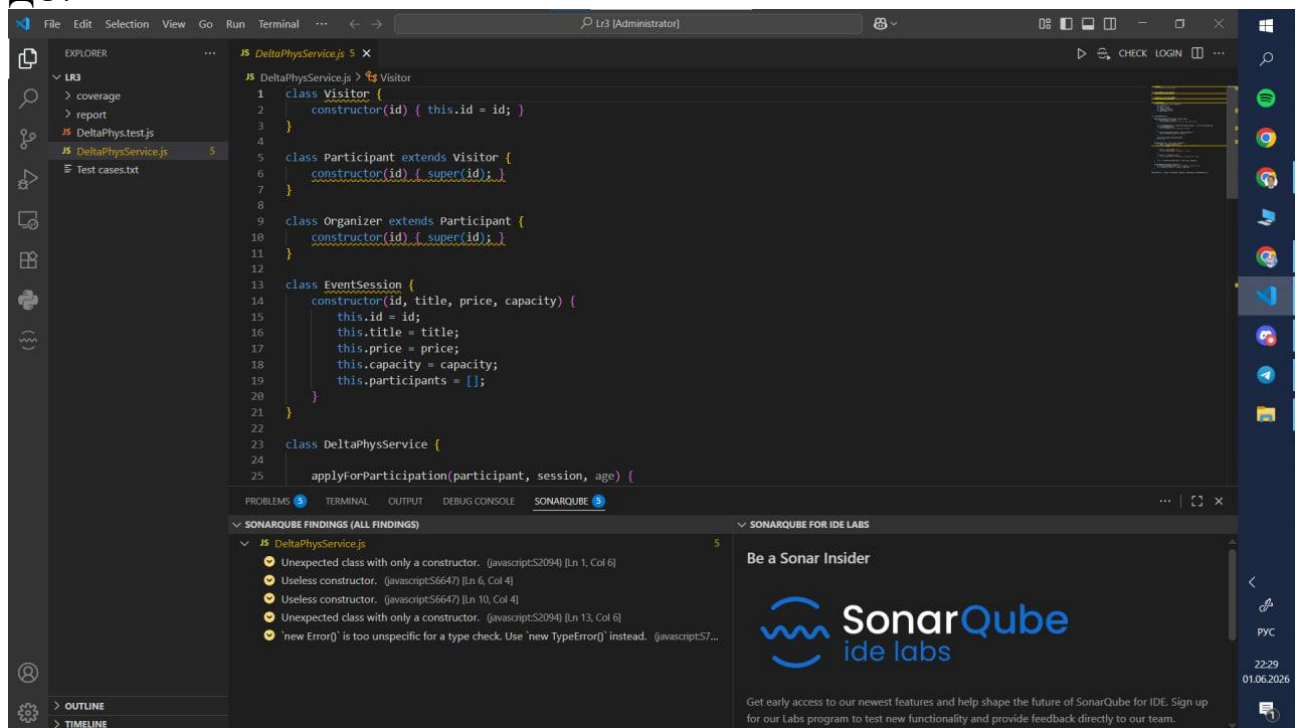
-----
File                                % Stmts  % Branch  % Funcs  % Lines  Uncovered Line #s
-----
All files                            100       100       100       100
DeltaPhysService.js                  100       100       100       100
-----

Test Suites: 1 passed, 10 total
Tests:       14 passed, 14 total
Snapshots:   0 total
Time:        1.154 s
ran all test suites. Ran 14 passed, 14 total
PS C:\Users\Anastasia\Desktop\ppor\OPT\LC3>
```

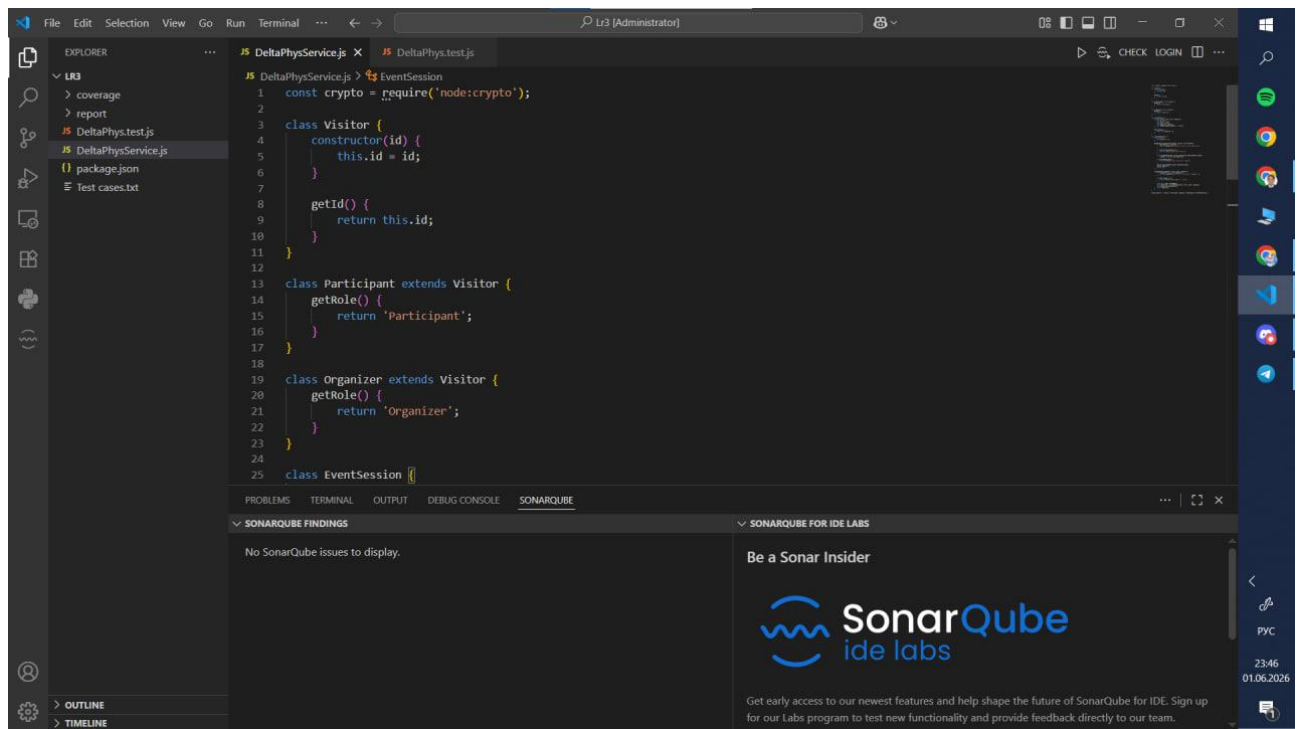
Порівняння метрик «ДО / ПІСЛЯ» :

SonarLint issues:

ДО:



ПІСЛЯ:



Цикломатична складність (за допомогою Lizard):

ДО:

```

1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
    66      5.4    1.6    45.2      10    DeltaPhysService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
    66      5.4    1.6    45.2      10          0    0.00  0.00
(.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3>

=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
    66      5.4    1.6    45.2      10    DeltaPhysService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
    66      5.4    1.6    45.2      10          0    0.00  0.00
(.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3>

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
    66      5.4    1.6    45.2      10          0    0.00  0.00
(.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3>
    66      5.4    1.6    45.2      10          0    0.00  0.00
    66      5.4    1.6    45.2      10          0    0.00  0.00
(.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3>

```

ПІСЛЯ:

```

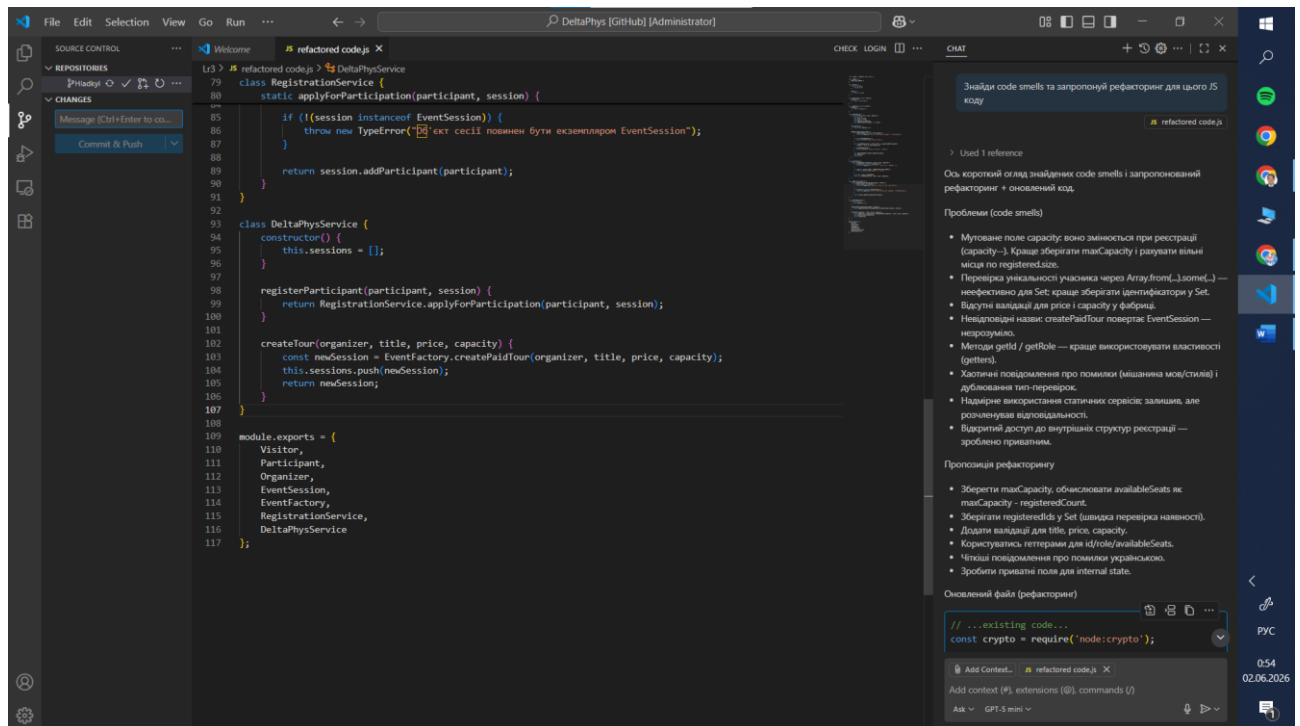
● (.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3> lizard DeltaPhysService.js
=====
NLOC    CCN  token  PARAM  length  location
-----
    3     1    11     1      3  constructor@4-6@DeltaPhysService.js
    3     1     9     0      3  getId@8-10@DeltaPhysService.js
    3     1     7     0      3  getRole@14-16@DeltaPhysService.js
    3     1     7     0      3  getRole@20-22@DeltaPhysService.js
    7     1    44     4      7  constructor@26-32@DeltaPhysService.js
    3     1    11     0      3  hasFreePlaces@34-36@DeltaPhysService.js
    3     1    11     0      3  constructor@40-42@DeltaPhysService.js
    1     1     8     0      1  (anonymous)@54-54@DeltaPhysService.js
   16     5    92     3     20  applyForParticipation@44-63@DeltaPhysService.js
   12     3    82     4     14  createPaidTour@65-78@DeltaPhysService.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
    65      5.4    1.6    28.2      10    DeltaPhysService.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
    65      5.4    1.6    28.2      10          0    0.00  0.00
○ (.venv) PS C:\Users\Anastasia\Desktop\npor\OPI\Lr3>

```


Бонусне завдання: Використання GitHub Copilot Chat

Я використав GitHub Copilot Chat, щоб перевірити свій код. ШІ знайшов помилку в класі EventSession. Виявилось, що я створив колекцію new Set(), але замість швидкого пошуку перетворив її назад у масив через Array.from().some(). Це уповільнювало роботу програми.



Висновок: Делегування логіки валідації місць до класу EventSession та заміна каскадних умовних операторів поліморфізмом дозволили суттєво знизити цикломатичну складність методів applyForParticipation та createPaidTour.

Підсумкова рефлексія Практична робота наочно продемонструвала, що Code Review є ефективним методом підвищення якості коду та обміну знаннями в команді. Завдяки рефакторингу на основі ООП-шаблонів (інкапсуляція, поліморфізм) та впровадженню Guard Clauses архітектура застосунку стала більш читабельною, гнучкою та безпечною. Крім того, робота підкреслила незамінність регресійного тестування: покриття коду юніт-тестами на Jest забезпечило надійний «страхувальний жест», що дозволило сміливо змінювати внутрішню логіку сервісів без ризику зламати наявний функціонал.